Big Data Analytics Lab Record

BIS701 — Hadoop MapReduce Experiments

Experiment 3: Weather Data Mining with MapReduce

Objective: Analyze weather data and display appropriate messages indicating weather conditions.

Java Implementation — WeatherAnalyzer.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import java.io.IOException;

public class WeatherAnalyzer {

    public static class WeatherMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            if (line.startsWith("Date") || line.isEmpty()) {
                return;
            }

            String[] fields = line.split(",");

            try {
                String date = fields[0];
                String year = date.substring(0, 4);
                int temperature = Integer.parseInt(fields[1].trim());
                int humidity = Integer.parseInt(fields[2].trim());
                String condition = fields[3].trim();

                context.write(new Text(year + "_TEMP"), new IntWritable(temperature));
                context.write(new Text(year + "_HUMIDITY"), new IntWritable(humidity));
                context.write(new Text(year + "_" + condition.toUpperCase()),
                            new IntWritable(1));

            } catch (Exception e) {
                System.err.println("Error parsing line: " + line);
            }
        }
    }
```

```
    public static class WeatherReducer extends Reducer<Text, IntWritable, Text, Text> {

        @Override
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {

            int sum = 0;
            int count = 0;
            int max = Integer.MIN_VALUE;
            int min = Integer.MAX_VALUE;

            for (IntWritable val : values) {
                int value = val.get();
                sum += value;
                count++;
                if (value > max) max = value;
                if (value < min) min = value;
            }

            String keyStr = key.toString();
            String message = "";

            if (keyStr.contains("_TEMP")) {
                double avg = (double) sum / count;
                message = String.format("Avg: %.2f\u00b0C, Max: %d\u00b0C, Min: %d\u00b0C |
",
                                        avg, max, min);

                if (avg > 35) {
                    message += "WARNING: Very Hot Weather!";
                } else if (avg > 25) {
                    message += "Warm Weather";
                } else if (avg > 15) {
                    message += "Pleasant Weather";
                } else if (avg > 5) {
                    message += "Cool Weather";
                } else {
                    message += "Cold Weather";
                }

            } else if (keyStr.contains("_HUMIDITY")) {
                double avg = (double) sum / count;
                message = String.format("Avg Humidity: %.2f%% | ", avg);

                if (avg > 80) {
                    message += "Very Humid";
                } else if (avg > 60) {
                    message += "Moderately Humid";
                } else {
                    message += "Dry";
                }

            } else {
                message = "Total days: " + count;
            }

            context.write(key, new Text(message));
        }
```

```
    }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Weather Analysis");

        job.setJarByClass(WeatherAnalyzer.class);
        job.setMapperClass(WeatherMapper.class);
        job.setReducerClass(WeatherReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Execution Steps

```
# Create input file
hdfs dfs -mkdir /weather
hdfs dfs -put weather_data.csv /weather/

# Compile
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath $HADOOP_CLASSPATH WeatherAnalyzer.java
jar -cvf weather.jar *.class

# Run
hadoop jar weather.jar WeatherAnalyzer /weather/weather_data.csv /weather/output

# View results
hdfs dfs -cat /weather/output/part-r-00000
```

Experiment 4: MovieLens Data Analysis — Finding Tags for Movies

Objective: Analyze MovieLens data to find tags associated with each movie.

Java Implementation — MovieTagAnalyzer.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
```

```
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.MultipleInputs;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import java.io.IOException;
import java.util.HashSet;
import java.util.Set;

public class MovieTagAnalyzer {

    public static class MovieMapper extends Mapper<LongWritable, Text, Text, Text> {

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            if (line.startsWith("movieId") || line.isEmpty()) {
                return;
            }

            try {
                String[] parts = line.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)");

                if (parts.length >= 2) {
                    String movieId = parts[0].trim();
                    String title = parts[1].trim().replace("\"", "");

                    context.write(new Text(movieId), new Text("M:" + title));
                }
            } catch (Exception e) {
                System.err.println("Error in MovieMapper: " + e.getMessage());
            }
        }
    }

    public static class TagMapper extends Mapper<LongWritable, Text, Text, Text> {

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();

            if (line.startsWith("userId") || line.isEmpty()) {
                return;
            }

            try {
                String[] parts = line.split(",");

                if (parts.length >= 3) {
                    String movieId = parts[1].trim();
                    String tag = parts[2].trim().replace("\"", "");

                    context.write(new Text(movieId), new Text("T:" + tag));
                }
```

```
            } catch (Exception e) {
                System.err.println("Error in TagMapper: " + e.getMessage());
            }
        }
    }

    public static class MovieTagReducer extends Reducer<Text, Text, Text, Text> {

        @Override
        public void reduce(Text key, Iterable<Text> values, Context context)
                throws IOException, InterruptedException {

            String movieTitle = "Unknown";
            Set<String> tags = new HashSet<>();

            for (Text val : values) {
                String value = val.toString();

                if (value.startsWith("M:")) {
                    movieTitle = value.substring(2);
                } else if (value.startsWith("T:")) {
                    tags.add(value.substring(2));
                }
            }

            if (!tags.isEmpty()) {
                String tagList = String.join(", ", tags);
                String output = String.format("%s | Tags: %s (Total: %d)",
                                              movieTitle, tagList, tags.size());
                context.write(key, new Text(output));
            }
        }
    }

    public static void main(String[] args) throws Exception {

        if (args.length != 3) {
            System.err.println("Usage: MovieTagAnalyzer <movies.csv> <tags.csv> <output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Movie Tag Analysis");

        job.setJarByClass(MovieTagAnalyzer.class);
        job.setReducerClass(MovieTagReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        MultipleInputs.addInputPath(job, new Path(args[0]),
                                    TextInputFormat.class, MovieMapper.class);
        MultipleInputs.addInputPath(job, new Path(args[1]),
                                    TextInputFormat.class, TagMapper.class);

        FileOutputFormat.setOutputPath(job, new Path(args[2]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
```

```
    }
}
```

Execution Steps

```
# Upload data to HDFS
hdfs dfs -mkdir /movielens
hdfs dfs -put movies.csv /movielens/
hdfs dfs -put tags.csv /movielens/

# Compile
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath $HADOOP_CLASSPATH MovieTagAnalyzer.java
jar -cvf movietag.jar *.class

# Run
hadoop jar movietag.jar MovieTagAnalyzer /movielens/movies.csv /movielens/tags.csv
/movielens/output

# View results
hdfs dfs -cat /movielens/output/part-r-00000
```